Light

Programming Fundamentals

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Variables, constants and data types
2. Operators and expressions
3. Selection and iteration
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Programming Fundamentals (1CP2/02 Topic 6: Problem Solving with Programming)**. This topic covers the essential building blocks of programming: variables, constants, data types, operators, input/output, and control structures. All code examples use Python 3 following the Edexcel Programming Language Subset (PLS).

Paper 2 is a practical onscreen exam where you write, trace, and debug Python code. You need to understand how to **declare variables**, **use appropriate data types**, **write expressions** using operators, and **control program flow** using selection and iteration. Expect questions requiring you to read code and predict output, complete partial programs, and fix errors.

Why programming fundamentals matter

These concepts are the foundation of all programming. Every program you write will use variables, data types, operators, and control structures:

- **Variables and constants:** Store and manage data that your program works with

- **Data types:** Ensure data is stored and processed correctly — numbers behave differently from text
- **Operators:** Perform calculations, make comparisons, and combine conditions
- **Selection:** Make decisions in your code — do different things based on conditions
- **Iteration:** Repeat actions efficiently without writing the same code multiple times
- **Exam relevance:** Paper 2 is entirely about programming. Every question requires understanding of these fundamentals

What you must know

This checklist covers the key programming concepts from Edexcel 1CP2 Topic 6. You must be able to read, write, trace, and debug code using these concepts.

- **Declare and use** variables with meaningful identifiers, and explain when to use constants
- **Identify and use** the four primitive data types: integer, real (float), Boolean, and string
- **Convert** between data types using `int()`, `float()`, `str()`, and `bool()`
- **Use** arithmetic operators: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- **Use** relational operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Use** logical operators: `and`, `or`, `not`
- **Write** selection statements using `if`, `elif`, and `else`
- **Write** iteration using `for` loops (count-controlled) and `while` loops (condition-controlled)
- **Use** `input()` and `print()` for user interaction
- **Apply** good coding practices: meaningful names, comments, and proper indentation

> **Specification link:** This topic covers Edexcel 1CP2 Paper 2 Topic 6 — Problem solving with programming. All code must follow the Programming Language Subset (PLS) which defines exactly which Python constructs are permitted in the exam.

Toolkit

These are the key code patterns and templates you need for programming questions.

Data types overview

Data type	Python name	Description	Example values
Integer	`int`	Whole numbers (positive, negative, or zero)	`42`, `-7`, `0`
Real / Float	`float`	Numbers with decimal points	`3.14`, `-0.5`, `2.0`
Boolean	`bool`	True or False values only	`True`, `False`
String	`str`	Text — sequence of characters	`"Hello"`, `'A'`, `"123"`

Arithmetic operators

Operator	Name	Example	Result
`+`	Addition	`7 + 3`	`10`
`-`	Subtraction	`7 - 3`	`4`
`*`	Multiplication	`7 * 3`	`21`
`/`	Division (real)	`7 / 3`	`2.333...`
`//`	Integer division (quotient)	`7 // 3`	`2`
`%`	Modulus (remainder)	`7 % 3`	`1`
`**`	Exponentiation (power)	`2 ** 3`	`8`

Relational operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`<`	Less than	`3 < 5`	`True`
`>`	Greater than	`3 > 5`	`False`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>=`	Greater than or equal to	`3 >= 5`	`False`

Logical operators

Operator	Meaning	Example	Result
`and`	Both conditions must be True	`True and False`	`False`
`or`	At least one condition must be True	`True or False`	`True`
`not`	Reverses the Boolean value	`not True`	`False`

Essential code templates

Variable declaration and assignment

```
name = "Alice"           # String variable
age = 16                 # Integer variable
price = 9.99             # Float variable
is_valid = True          # Boolean variable
MAX_ATTEMPTS = 3         # Constant (uppercase by convention)
```

Input and output

```
# Getting input from the user
name = input("Enter your name: ")
age = int(input("Enter your age: "))      # Convert to integer
price = float(input("Enter price: "))     # Convert to float

# Displaying output
print("Hello", name)
print("You are", age, "years old")
print("Total:", price * 1.2)
```

Selection (if/elif/else)

```
# Simple if statement
if score >= 50:
    print("Pass")

# if-else statement
if age >= 18:
    print("Adult")
else:
    print("Minor")

# if-elif-else statement
if grade >= 70:
    print("Distinction")
elif grade >= 50:
    print("Merit")
elif grade >= 40:
    print("Pass")
else:
    print("Fail")
```

Iteration (loops)

```
# For loop — count-controlled (repeat a set number of times)
for i in range(5):           # i = 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):        # i = 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2):    # i = 0, 2, 4, 6, 8 (step of 2)
    print(i)

# While loop — condition-controlled (repeat until condition is false)
count = 0
while count < 5:
    print(count)
    count = count + 1
```

1. Variables, constants and data types

Variables and constants are used to store data in a program. Understanding data types ensures your program handles data correctly.

1.1 Variables

A **variable** is a named location in memory that stores a value. The value can change during program execution.

Declaring variables in Python:

```
score = 0           # Create variable, assign value 0
score = score + 10  # Change value to 10
score = 25          # Change value to 25
```

Rules for variable names (identifiers):

- Must start with a letter or underscore
- Can contain letters, numbers, and underscores
- Cannot contain spaces or special characters
- Cannot be a Python keyword (like `if`, `for`, `while`)
- Are case-sensitive (`Score` and `score` are different)

Good practice — meaningful names:

```
# Good — clear what the variable represents
player_score = 100
total_price = 49.99
user_name = "Alex"

# Bad — unclear or confusing
x = 100
tp = 49.99
n = "Alex"
```

> **Exam tip:** Always use meaningful variable names. Marks can be lost for using single letters like `x` when the purpose is unclear.

1.2 Constants

A **constant** is a value that does not change during program execution. In Python, constants are created by convention using UPPERCASE names.

```
MAX_ATTEMPTS = 3
VAT_RATE = 0.2
PI = 3.14159
```

Why use constants?

- Makes code easier to read and understand
- Easy to update — change in one place, affects whole program
- Prevents accidental changes to important values
- Documents the purpose of "magic numbers"

```
# Without constant — what does 0.2 mean?
total = price + (price * 0.2)

# With constant — clear purpose
VAT_RATE = 0.2
total = price + (price * VAT_RATE)
```

1.3 Data types

Every value in Python has a **data type** that determines what operations can be performed on it.

The four primitive data types:

- **Integer (int):** Whole numbers — `42`, `-7`, `0`
- **Float (real):** Decimal numbers — `3.14`, `-0.5`
- **Boolean (bool):** `True` or `False`
- **String (str):** Text in quotes — `"Hello"`, `'A'`

> **Exam tip:** The Edexcel spec uses "real" to mean float. Both terms refer to decimal numbers.

1.4 Type conversion (casting)

Sometimes you need to convert data from one type to another. This is called **type conversion** or **casting**.

```
# String to integer
age_str = "25"
age = int(age_str)        # age = 25 (integer)

# String to float
price_str = "19.99"
price = float(price_str)  # price = 19.99 (float)

# Number to string
score = 100
score_str = str(score)    # score_str = "100" (string)

# Integer to float
num = 5
num_float = float(num)    # num_float = 5.0
```

Why is type conversion needed?

The `input()` function always returns a string. If you want to do calculations, you must convert to a number first:

```
# This causes an error or wrong result:
age = input("Enter age: ")
next_year = age + 1       # Error! Can't add string + int

# Correct approach:
age = int(input("Enter age: "))
next_year = age + 1       # Works correctly
```

> **Exam tip:** A very common error is forgetting to convert input to a number. Always use `int()` or `float()` when you need to do maths with user input.

1.5 Comments

Comments are notes in your code that are ignored by Python. They help explain what the code does.

```
# This is a single-line comment

score = 0  # Initialise score to zero

# Calculate the total with VAT
# VAT rate is 20%
total = price * 1.2
```

Good comments explain **why** something is done, not just what it does. The code itself shows what happens; comments explain the reasoning.

Worked Example 1 – Identifying data types

Question: State the most appropriate data type for each of the following. [4 marks]
(a) A person's age in whole years
(b) The price of a product
(c) Whether a user is logged in
(d) A customer's name

Model answer:
(a) Integer **(1)** — age is a whole number
(b) Float / Real **(1)** — prices have decimal places
(c) Boolean **(1)** — only two states: logged in or not
(d) String **(1)** — names are text

Exam tip: Choose the most appropriate type. A phone number should be a string (not integer) because you don't do maths with it and it might have leading zeros.

Worked Example 2 – Type conversion

Question: What is the output of the following code? [2 marks]

```
num1 = "5"
num2 = "3"
print(num1 + num2)
```

Model answer:
Output: `53` **(1)**
Because both values are strings, the `+` operator concatenates (joins) them rather than adding them mathematically **(1)**.

Exam tip: The + operator behaves differently for strings (concatenation) and numbers (addition). Always check the data type.

Worked Example 3 — Fixing a type error

Question: The following code is intended to add two numbers entered by the user, but it produces the wrong result. Identify the error and write the corrected code. [3 marks]

```
num1 = input("Enter first number: ")
num2 = input("Enter second number: ")
total = num1 + num2
print("Total:", total)
```

Model answer:

Error: `input()` returns strings, so `num1 + num2` concatenates instead of adding **(1)**.

Corrected code:

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
total = num1 + num2
print("Total:", total)
```

The inputs must be converted to integers using `int()` **(2)**.

Exam tip: This is one of the most common errors in Paper 2. Always convert input when you need to do calculations.

Worked Example 4 — Using constants

Question: Explain one benefit of using constants in a program. [2 marks]

Model answer:

Constants make the code easier to maintain **(1)**.

If the value needs to change, you only update it in one place rather than searching through the whole program **(1)**.

Alternative answers:

- Makes code more readable — the constant name explains what the value represents
- Prevents accidental changes to important values

> **Exam tip:** Link your answer — state the benefit AND explain why it is a benefit.

2. Operators and expressions

Operators perform operations on values. An **expression** is a combination of values, variables, and operators that evaluates to a result.

2.1 Arithmetic operators in detail

Arithmetic operators perform mathematical calculations.

Division operators

Python has two division operators that often cause confusion:

```
# Real division (/) — gives decimal result
print(7 / 2)    # Output: 3.5
print(10 / 4)   # Output: 2.5
print(6 / 3)    # Output: 2.0 (still a float!)

# Integer division (//) — gives whole number only
print(7 // 2)   # Output: 3
print(10 // 4)  # Output: 2
print(6 // 3)   # Output: 2
```

Modulus operator (%)

The modulus operator gives the **remainder** after division:

```
print(7 % 3)    # Output: 1 (7 ÷ 3 = 2 remainder 1)
print(10 % 4)   # Output: 2 (10 ÷ 4 = 2 remainder 2)
print(15 % 5)   # Output: 0 (15 ÷ 5 = 3 remainder 0)
```

Common uses of modulus:

- Check if a number is even: `num % 2 == 0`
- Check if a number is divisible by another: `num % 5 == 0`
- Get the last digit: `num % 10`
- Wrap around (clock arithmetic): `hour % 12`

> **Exam tip:** Integer division (`//`) and modulus (`%`) are very commonly tested. Practice using them together.

2.2 Operator precedence

When an expression has multiple operators, Python follows **precedence rules** (like BIDMAS in maths):

Priority	Operators
1 (highest)	`()` — Brackets
2	`**` — Exponentiation
3	`*`, `/`, `//`, `%`

Priority	Operators
4	`+, -`
5	`==, !=, <, >, <=, >=`
6	`not`
7	`and`
8 (lowest)	`or`

```
# Precedence examples
print(2 + 3 * 4)      # 14 (not 20)
print((2 + 3) * 4)    # 20 (brackets first)
print(10 - 4 - 2)     # 4 (left to right)
print(2 ** 3 ** 2)    # 512 (right to left for **)
```

Exam tip: When in doubt, use brackets to make your intention clear. It also makes code easier to read.

2.3 Relational operators

Relational operators compare two values and return a Boolean (`True` or `False`).

```
# Comparing numbers
print(5 == 5)     # True — equal to
print(5 != 3)     # True — not equal to
print(3 < 5)      # True — less than
print(3 > 5)      # False — greater than
print(5 <= 5)     # True — less than OR equal to
print(3 >= 5)     # False — greater than OR equal to

# Comparing strings (alphabetical order)
print("apple" < "banana")   # True
print("Zoo" < "apple")      # True (uppercase comes before lowercase)
```

Common mistake: Using = instead of ==. A single = is assignment; double == is comparison.

2.4 Logical operators

Logical operators combine Boolean values or expressions.

AND — both must be True

A	B	A and B
True	True	True
True	False	False
False	True	False

A	B	A and B
False	False	False

```
age = 25
has_licence = True
if age >= 17 and has_licence:
    print("Can drive")
```

OR — at least one must be True

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

```
day = "Saturday"
if day == "Saturday" or day == "Sunday":
    print("Weekend!")
```

NOT — reverses the value

```
logged_in = False
if not logged_in:
    print("Please log in")
```

Worked Example 5 — Integer division and modulus

Question: What is the output of the following code? [2 marks]

```
minutes = 135
hours = minutes // 60
remaining = minutes % 60
print(hours, "hours and", remaining, "minutes")
```

Model answer:

`hours = 135 // 60 = 2`

`remaining = 135 % 60 = 15`

Output: `2 hours and 15 minutes` **(2)**

Exam tip: This pattern (using `//` and `%` together) is very common for converting between units like minutes/hours or pence/pounds.

Worked Example 6 — Evaluating expressions

Question: Evaluate the following expressions. [4 marks]

(a) `3 + 4 * 2`
(b) `15 // 4`
(c) `17 % 5`
(d) `2 ** 4`

Model answer:

(a) `3 + 4 * 2 = 3 + 8 = 11` **(1)** — multiplication before addition
(b) `15 // 4 = 3` **(1)** — integer division gives whole number
(c) `17 % 5 = 2` **(1)** — 17 ÷ 5 = 3 remainder 2
(d) `2 ** 4 = 16` **(1)** — 2 to the power of 4

Exam tip: Show your working for operator precedence questions. Write out the steps in order.

Worked Example 7 — Logical operators

Question: What is the output of the following code? [2 marks]

```
x = 10
y = 5
print(x > 5 and y < 10)
print(x < 5 or y > 3)
```

Model answer:

Line 3: `x > 5` is `True`, `y < 10` is `True`, so `True and True = True` **(1)**
Line 4: `x < 5` is `False`, `y > 3` is `True`, so `False or True = True` **(1)**

Output:

```
True True
```

Exam tip: Evaluate each comparison first, then apply the logical operator.

Worked Example 8 — Writing expressions

Question: Write a Python expression to check if a variable `age` is between 13 and 19 inclusive. [2 marks]

Model answer:

```
age >= 13 and age <= 19
```

Both conditions must be true, so use `and` **(1)**.
Use `>=` and `<=` to include the boundary values **(1)**.

Exam tip: "Inclusive" means including the boundary values. Use `>=` and `<=`. "Exclusive" means not including them — use `>` and `<`.

3. Selection and iteration

Selection and iteration are **control structures** that determine the order in which statements are executed.

3.1 Selection (if statements)

Selection allows the program to make decisions and execute different code based on conditions.

Simple if statement

```
if condition:
    # Code runs only if condition is True
    statement
```

if-else statement

```
if condition:
    # Runs if condition is True
    statement1
else:
    # Runs if condition is False
    statement2
```

if-elif-else statement

```
if condition1:
    statement1
elif condition2:
    statement2
elif condition3:
    statement3
else:
    statement4
```

Conditions are checked in order. As soon as one is True, its block runs and the rest are skipped.

> **Exam tip:** Indentation is essential in Python. Code inside an `if` block must be indented consistently.

3.2 Nested selection

Selection statements can be **nested** inside each other:

```
age = int(input("Enter age: "))
has_ticket = input("Have ticket? (y/n): ")

if age >= 18:
    if has_ticket == "y":
        print("Entry allowed")
    else:
        print("Buy a ticket first")
else:
    print("Must be 18 or over")
```

Trace through with age = 20, has_ticket = "n":

1. Check `age >= 18`: 20 >= 18 is True
2. Enter the first if block
3. Check `has_ticket == "y"`: "n" == "y" is False
4. Execute the else block: print "Buy a ticket first"

> **Alternative approach:** Nested ifs can often be rewritten using `and`:
>
> ```
> if age >= 18 and has_ticket == "y":
>  print("Entry allowed")
> elif age >= 18:
>  print("Buy a ticket first")
> else:
>  print("Must be 18 or over")
> ```

3.3 Iteration – for loops

A **for loop** is **count-controlled** — it repeats a set number of times.

Basic for loop with range()

```
# range(n) gives values 0, 1, 2, ..., n-1
for i in range(5):
    print(i)
```

```
0 1 2 3 4
```

range() with start and stop

```
# range(start, stop) – stop is NOT included
for i in range(1, 6):
    print(i)
```

```
1 2 3 4 5
```

range() with step

```
# range(start, stop, step)
for i in range(0, 10, 2):
    print(i)
```

```
0 2 4 6 8
```

Counting backwards

```
for i in range(5, 0, -1):
    print(i)
```

```
5 4 3 2 1
```

> **Exam tip:** Remember that `range(n)` produces n values from 0 to n-1, not 1 to n. The stop value is never included.

3.4 Iteration — while loops

A **while loop** is **condition-controlled** — it repeats while a condition is True.

Basic while loop

```
count = 0
while count < 5:
    print(count)
    count = count + 1
```

```
0 1 2 3 4
```

Structure:

1. Check condition — if False, exit loop
2. Execute loop body
3. Return to step 1

Input validation with while

```
# Keep asking until valid input
password = ""
while password != "secret":
    password = input("Enter password: ")
print("Access granted")
```

Sentinel value

```
# Loop until user enters -1
total = 0
num = int(input("Enter number (-1 to stop): "))
while num != -1:
    total = total + num
    num = int(input("Enter number (-1 to stop): "))
print("Total:", total)
```

Infinite loops: If the condition never becomes False, the loop runs forever. Always ensure something in the loop can change the condition.

```
# Infinite loop — count never changes!
count = 0
while count < 5:
    print(count)
    # Missing: count = count + 1
```

3.5 Choosing between for and while

Use for loop when...	Use while loop when...
You know how many times to repeat	You don't know how many times to repeat
Iterating through a range of numbers	Repeating until a condition is met
Processing each item in a list	Validating user input
Example: "Print numbers 1 to 10"	Example: "Keep asking until correct"

Worked Example 9 — Tracing selection (if-elif-else)

Question: What is the output of the following code when the user enters 72? [3 marks]

```
score = int(input("Enter score: "))
if score >= 90:
    print("Grade A")
elif score >= 70:
    print("Grade B")
elif score >= 50:
    print("Grade C")
else:
    print("Grade F")
```

Model answer:

Input: score = 72

Check `score >= 90`: 72 >= 90 is False — skip this block **(1)**

Check `score >= 70`: 72 >= 70 is True — execute this block **(1)**

Output: `Grade B` **(1)**
Note: Once a condition is True, remaining elif/else blocks are skipped.

Exam tip: Work through each condition in order. As soon as one is True, that block runs and the rest are skipped — even if later conditions would also be True.

Worked Example 10 — Tracing a for loop

Question: What is the output of the following code? [3 marks]

```
total = 0
for i in range(1, 5):
    total = total + i
    print(i, total)
```

Model answer:
Trace table:

i	total (before)	total (after)	Output
1	0	1	1 1
2	1	3	2 3
3	3	6	3 6
4	6	10	4 10

Output: (3)

```
1 1 2 3 3 6 4 10
```

Exam tip: Use a trace table to track variable values through each iteration. This helps avoid mistakes.

Worked Example 11 — Tracing a while loop

Question: What is the output of the following code? [3 marks]

```
x = 10
while x > 0:
    print(x)
    x = x - 3
```

Model answer:
Trace through:

- x = 10: 10 > 0 is True, print 10, x becomes 7
- x = 7: 7 > 0 is True, print 7, x becomes 4
- x = 4: 4 > 0 is True, print 4, x becomes 1
- x = 1: 1 > 0 is True, print 1, x becomes -2
- x = -2: -2 > 0 is False, exit loop

Output: (3)

```
10 7 4 1
```

Exam tip: Always check the condition BEFORE each iteration. The loop stops when the condition first becomes False.

Worked Example 12 — Writing a for loop

Question: Write a Python program that displays all even numbers from 2 to 20 inclusive. [3 marks]

Model answer:

```
for i in range(2, 21, 2):
    print(i)
```

Use `range(2, 21, 2)`: start at 2, stop before 21, step by 2 **(3)**

Alternative answer:

```
for i in range(2, 21):
    if i % 2 == 0:
        print(i)
```

Exam tip: Remember that the stop value in range() is exclusive, so use 21 to include 20.

Worked Example 13 — Writing a while loop for validation

Question: Write a Python program that asks the user for a number between 1 and 100 and keeps asking until a valid number is entered. [4 marks]

Model answer:

```
num = int(input("Enter a number between 1 and 100: "))
while num < 1 or num > 100:
    print("Invalid input")
    num = int(input("Enter a number between 1 and 100: "))
print("You entered:", num)
```

(1) Initial input
(1) Correct condition using `or`
(1) Error message in loop
(1) Re-prompt for input in loop

Exam tip: For "not in range" validation, use `or`: the number is invalid if it's less than the minimum OR greater than the maximum.

Worked Example 14 — Fixing a loop error

Question: The following code is intended to print the numbers 1 to 5, but it contains an error. Identify and fix the error. [2 marks]

```
for i in range(5):
    print(i)
```

Model answer:
Error: `range(5)` produces 0, 1, 2, 3, 4 — not 1 to 5 **(1)**
Fix: Change to `range(1, 6)` to produce 1, 2, 3, 4, 5 **(1)**

```
for i in range(1, 6):
    print(i)
```

Exam tip: Off-by-one errors are very common. Always check the start and end values carefully.

Worked Example 15 — Complete program with selection and iteration

Question: Write a Python program that asks the user for 5 test scores, calculates the average, and displays whether the average is a pass (50 or above) or fail. [6 marks]

Model answer:

```
total = 0
for i in range(5):
```

```
    score = int(input("Enter score: "))
    total = total + score
average = total / 5
print("Average:", average)
if average >= 50:
    print("Pass")
else:
    print("Fail")
```

(1) Initialise total to 0
(1) Correct for loop for 5 iterations
(1) Input and convert score inside loop
(1) Accumulate total
(1) Calculate average after loop
(1) Correct if-else for pass/fail

Exam tip: For multi-mark programming questions, break the problem into steps. Get each part working before moving to the next.

Worked examples summary

This topic includes 15 worked examples covering the key programming skills you need for Paper 2.

Example	Skill tested	Topic
1	Identifying appropriate data types	Data types
2	Understanding string concatenation	Data types
3	Fixing type conversion errors	Type conversion
4	Explaining benefits of constants	Constants
5	Using integer division and modulus	Operators
6	Evaluating arithmetic expressions	Operators
7	Evaluating logical expressions	Operators
8	Writing compound conditions	Operators
9	Tracing if-elif-else selection	Selection
10	Tracing a for loop	Iteration
11	Tracing a while loop	Iteration
12	Writing a for loop with range()	Iteration
13	Writing a while loop for validation	Iteration

Example	Skill tested	Topic
14	Fixing off-by-one loop errors	Iteration
15	Complete program with loops and selection	Combined

Quick reference

Key terms

Term	Definition
Variable	A named location in memory that stores a value which can change during program execution
Constant	A named value that does not change during program execution (written in UPPERCASE by convention)
Data type	A classification that determines what kind of value a variable can hold and what operations can be performed
Integer	A whole number data type (positive, negative, or zero)
Float / Real	A decimal number data type
Boolean	A data type with only two possible values: True or False
String	A sequence of characters (text) enclosed in quotes
Type conversion / Casting	Converting a value from one data type to another
Selection	A control structure that executes different code based on a condition (if statements)
Iteration	A control structure that repeats code (loops)
Count-controlled loop	A loop that repeats a set number of times (for loop)
Condition-controlled loop	A loop that repeats while a condition is true (while loop)

Python syntax quick reference

Task	Python syntax
Assign a variable	`name = value`
Get user input	`variable = input("prompt")`

Task	Python syntax
Get integer input	`variable = int(input("prompt"))`
Display output	`print(value)` or `print(value1, value2)`
Comment	`# This is a comment`
If statement	`if condition:`
If-else statement	`if condition: ... else:`
If-elif-else	`if: ... elif: ... else:`
For loop (n times)	`for i in range(n):`
For loop (a to b)	`for i in range(a, b+1):`
While loop	`while condition:`

Common patterns

Running total (accumulator)

```
total = 0
for i in range(5):
    num = int(input("Enter number: "))
    total = total + num
print("Total:", total)
```

Counting occurrences

```
count = 0
for i in range(10):
    num = int(input("Enter number: "))
    if num > 0:
        count = count + 1
print("Positive numbers:", count)
```

Finding maximum

```
maximum = int(input("Enter first number: "))
for i in range(4):
    num = int(input("Enter number: "))
    if num > maximum:
```

```
        maximum = num
print("Maximum:", maximum)
```

Input validation

```
num = int(input("Enter 1-10: "))
while num < 1 or num > 10:
    print("Invalid")
    num = int(input("Enter 1-10: "))
print("Valid:", num)
```

Typical pitfalls

These are common mistakes students make in programming questions. Avoiding these errors will help you gain marks in Paper 2.

Pitfall 1 — Forgetting to convert input.
Wrong: `age = input("Enter age: ")` then doing maths
Right: `age = int(input("Enter age: "))` — always convert when doing calculations.

Pitfall 2 — Using = instead of ==.
Wrong: `if x = 5:`
Right: `if x == 5:` — single `=` is assignment, double `==` is comparison.

Pitfall 3 — Off-by-one errors with range().
Wrong: `range(1, 10)` to get 1-10
Right: `range(1, 11)` — the stop value is NOT included.

Pitfall 4 — Missing colon after if/for/while.
Wrong: `if x > 5`
Right: `if x > 5:` — colons are required before indented blocks.

Pitfall 5 — Incorrect indentation.
Wrong: Code that should be inside a block is not indented
Right: All code inside if/for/while must be consistently indented.

Pitfall 6 — Confusing / and // division.
Wrong: Using `/` when you need a whole number result
Right: Use `//` for integer division. `7/2 = 3.5` but `7//2 = 3`.

Pitfall 7 — Wrong logical operator in validation.
Wrong: `while num < 1 and num > 10` (impossible!)
Right: `while num < 1 or num > 10` — use OR for "outside range" checks.

Pitfall 8 — Infinite loops.
Wrong: While loop where the condition never becomes False
Right: Ensure something in the loop changes the condition (update counter, get new input).

Pitfall 9 — Comparing strings with numbers.
Wrong: `if age > 17:` when age is a string
Right: `if int(age) > 17:` — convert to number first for numeric comparison.

Pitfall 10 — Case sensitivity.
Wrong: Assuming `"Yes"` equals `"yes"`
Right: Python is case-sensitive. Use `.lower()` or check both cases.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.